

Python Code

```
1  ###Q2: Parameter Server###
2  ###please implement PS method, using parameter update&optimizer states all be held in
   rank0###
3
4  from torch.utils import _flatten_dense_tensors, _unflatten_dense_tensors
5  import torch
6  import torch.distributed as dist
7
8  def server(params, opt, world):
9      # ---- aggregate grads from workers ----
10     flat_grad = _flatten_dense_tensors([p.grad for p in params]).contiguous() #usage:
    transfer a list tensor to one 1-D tensor
11     ##here, you should generate one big 1-D tensor containing all parameters to make
    the transfer process easy
12     agg = flat_grad.clone() #agg as a aggregated counter to record sum gradients
13
14     recv_buf = torch.zeros_like(flat_grad)
15     for i in range(1, world):
16         r = dist.irecv(recv_buf, src=i)
17         r.wait()
18         agg.add_(recv_buf)
19
20     agg.div_(world)
21     synced_grads = _unflatten_dense_tensors(agg, [p.grad for p in params])
22     # ---- set averaged grads locally & step ----
23     for g, s in zip([p.grad for p in params], synced_grads):
24         g.copy_(s)
25     opt.step()
26
27     # ---- broadcast updated params for this subset ----
28     flat_param = _flatten_dense_tensors([p.data for p in params]).contiguous()
29                                     #
30     for dst in range(1, world):
31         s = dist.isend(flat_param, dst=dst)
32         s.wait()
33
34
35 def worker(params):
36     flat_grad = _flatten_dense_tensors([p.grad for p in params]).contiguous()
37     # ---- push grads to server ----
38     s = dist.isend(flat_grad, dst=0)
39     s.wait()
40
41     # ---- receive updated params, write into local model ----
42
43     flat_param_template = _flatten_dense_tensors([p.data for p in
    params]).contiguous()
44     recv_buf = torch.zeros_like(flat_param_template)
45
46     r = dist.irecv(recv_buf, src=0)
```

```
47     r.wait()
48
49     synced_params = _unflatten_dense_tensors(recv_buf, [p.data for p in params]) #you
should assign correct value for synced_params#
50     # ---- synchronize the parameters ----
51     for p, s in zip(params, synced_params):
52         p.data.copy_(s)
53
54 def PS_grads_(model, world_size=None, rankid=None, opt=None):
55     """
56     Synchronous PS step:
57     - Rank 0: receive grads, sum/avg, set grads, opt.step(), broadcast updated
params.
58     - Rank >0: send grads, receive updated params, write into local model.
59     Only processes the subset of parameters with non-None grads.
60     """
61     world = world_size
62     rank = rankid
63
64     # Fast path: single process
65     if world == 1:
66         opt.step()
67         return
68
69     #not necessary, as in most case, params won't be empty list...
70     # Collect params that participated in this backward pass
71     params = [p for p in model.parameters() if p.grad is not None]
72     if not params:
73         # No grads this step; only server might still want to advance schedulers, etc.
74         if rank == 0:
75             opt.step()
76             return
77
78     if rank == 0:
79         server(params, opt, world)
80
81     else:
82         worker(params)
83
84     # Optional hard step boundary
85     dist.barrier()
```